Phi Token Protocol Security Review

Reviewed by: MrPotatoMagic

October 2nd, 2025

Contents

1 About MrPotatoMagic . . . 5
2 Disclaimer . . . 5
3 Introduction . . . 5
4 Risk Classification . . . 5
4.1 Impact . . . 5
4.2 Likelihood . . . 5
4.3 Action required for severity levels . . . 6
5 Executive Summary . . . 6
6 Findings . . . 7
6.1 Critical Risk . . . 7
6.1.1 PhiEth creator can steal user deposited funds through Morpho Strategy . . . 7
6.1.2 Creator performing premine-zap unintentionally pre-closes fair launch positions . . . 8
6.1.3 Attacker can force all fair launches to fail by purchasing all tokens and selling them . . . 9
6.1.4 Attacker can steal all fair launch supply at no cost . . . 10
6.1.5 Board tokens can be permanently locked in BoardHook.sol contract . . . 10
6.2 High Risk . . . 11
6.2.1 Attacker can block creators from changing strategies through minimal donations . . . 11
6.2.2 Attacker can manipulate reserves on Aerodrome pools to cause slippage on withdrawAndSwap() . . . 12
6.2.3 Unspent swap tokens are locked in ReferralEscrow.sol . . . 13
6.2.4 Morpho withdrawal can fail due to rounding issue in max withdrawal check . . . 13
6.2.5 Extremely low INCREASE_TOKEN_VOLUME_THRESHOLD and MAX_VOLUME_THRESHOLD in DynamicFeeCalculator.sol leads to maximum fees . . . 14
6.3 Medium Risk . . . 15
6.3.1 Changing strategies will never be possible due to initialization check . . . 15
6.3.2 Changing strategies is not possible due to external initialize call . . . 15

6.3.3 Attacker can DOS publiphi by frontrunning boardIds 16
6.3.4 Fees are captured twice in both afterSwap and beforeSwap hooks 16
6.3.5 Attacker can steal creator's premine zap if creator directly uses publiphi on BoardHook contract 17
6.3.6 Missing slippage protection in PoolSwap.sol 17
6.3.7 Missing validation for PhiEth allows anyone to use a malicious/non-whitelisted PhiETH token when deploying strategies 19
6.3.8 Whitelisting in paused state is not enforced correctly 20
6.3.9 Rebalance ratio not respected when rebalance threshold is increased 20
6.3.10 Possibility of stuck Morpho reward tokens if reward token is fee-on-transfer 21
6.3.11 Missing access control on setFactory() enables frontrunning initialization 21
6.3.12 Inconsistent PhiEthDeployer update logic across contracts . 22
6.3.13 Ranking of existing second and third place top contributors does not increase 22
6.3.14 Contributor ranking logic incorrectly switches second rank instead of 3rd rank 23
6.3.15 Fees in feeSplit() are incorrectly calculated 24
6.3.16 Only first swap in block receives decreased volume based fee 25
6.3.17 Malicious user can dilute fees by providing 20 gwei priority fee at the start of block 26
6.3.18 Possible for gasBasedFeeScaled to be greater than maxGasFee in DynamicFeeCalculator 26
6.3.19 calculateGasBasedFee() incorrectly calculates gas based fee component for first swap in a block 27
6.3.20 Attacker can use flashswaps to increase fees for users 27
6.3.21 Fee decay does not factor in frequency of swaps 27
6.4 Low Risk 28
6.4.1 Remove console.log statement 28
6.4.2 Incorrect publiphiAt timestamp emitted on board creation . 28
6.4.3 Missing address(0) check when adding and removing a router 29
6.4.4 Missing event emission in setGasFeeParameters() in DynamicFeeCalculator.sol 29
6.4.5 Whitelist merkle root cannot be updated by creator 29
6.4.6 Protocol fee recipient can revert to temporarily DOS publiphi 30
6.4.7 Incorrect event emission of PoolFeesDistributed 30
6.4.8 Self referrals are possible 31
6.4.9 Incorrect event emission of TokenSwapped in ReferralEscrow.sol 31
6.4.10 Incorrect endsAt data in event FairLaunchEnded emission . 31
6.4.11 Function claimBoardAndSwap() can revert during fair launch window 32

6.4.12 Consider making slippage percentage configurable in AerodromeSwapper.sol . . . 33
6.4.13 Precision loss due to rounding down in withdrawAndSwap() 33
6.4.14 No incentive for users to purchase PhiEth . . . 33
6.4.15 Missing functionality to allow users to invalidate permits . . 34
6.4.16 Consider removing redundant zero address check in mint() . 34
6.4.17 Remove redundant functionality from Board.sol . . . 34
6.4.18 Inspirations to address(0) occur on burns . . . 36
6.4.19 Inspirer update logic allows creator to deprive original inspirer of fees . . . 36
6.4.20 Incorrect event emission details . . . 37
6.4.21 Incorrect fairlaunch comment in publiphi . . . 37
6.4.22 Redundant PhiEth token whitelist functionality in PhiEthFactory.sol . . . 37
6.4.23 Variable accumulatorLastUpdateTime updated even when accumulator weighted volume remains unchanged . . . 38
6.5 Non-Critical . . . 38
6.5.1 phiEthFactory is already assigned in FeeDistributor.sol . . . 38
6.5.2 Redundant else-if block in function modifyRevenue() . . . 38
6.5.3 Missing functionality for owner to update boardManager, feeExemptions, fairLaunch and notifier contract addresses . . . 39
6.5.4 Return value of createPosition() is never used . . . 40
6.5.5 Add premine zap contract to excludedAddresses mapping in MuseManager.sol . . . 40
6.5.6 Consider making swap() functions in the PoolSwap.sol contract non-payable . . . 40
6.5.7 Deadline is not required in withdrawAndSwap() . . . 40
6.5.8 Correct comment in PhiEthImpl . . . 41
6.5.9 Consider removing redundant variable from PhiEthFactory . 41
6.5.10 Consider introducing functionality to remove strategies from whitelist . . . 41
6.5.11 Function setBoardImplementation() can be optimized . . . 42
6.5.12 Pack struct members to save gas . . . 42
6.5.13 Consider removing redundant modifier from swap() . . . 42
6.5.14 Remove unused variable PhiEthDeployer . . . 43
6.5.15 Strategy implementations cannot be updated in strategy deployer . . . 43
6.5.16 Creator cannot deploy PhiEth token again with the same name and symbol . . . 44
6.5.17 Consider using named mappings in MuseManager.sol . . . 44
6.5.18 Typo error with inspirer instead of inspired . . . 44
6.5.19 Save gas by emitting event first . . . 45
6.5.20 Remove redundant excludedAddresses check from tryInspire() 45
6.5.21 Alias account transfers can be used to inspire . . . 46
6.5.22 Incorrect comments in trackSwap() . . . 46

1 About MrPotatoMagic

MrPotatoMagic is an independent smart contract security researcher. Having disclosed over 100 security vulnerabilities across numerous protocol types, he consistently aims to elevate the security of the blockchain ecosystem by offering top quality smart contract security reviews.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where I try to find as many vulnerabilities as possible. I can not guarantee 100 percent security after the review or even if the review will find any problems with your smart contracts. Subsequent testing, security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3 Introduction

Phi Labs has forged a strategic alliance with the top DeFi protocols in the space. By leveraging Uniswap V4, users can launch multiple coins for their Boards to tokenize onchain profiles and earn ETH yield powered by Aave and Morpho Labs to generate a consistent revenue stream. Board launches ensure a fair and equitable experience by avoiding insider allocations and preferential treatment. It also plays a pivotal and catalytic role in fuelling up the existing PHI ecosystem to enable sustained growth.

4 Risk Classification

Severity Level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- High - Leads to significant material loss of assets in the protocol or significant harm to majority of users.
- Medium - Loss/Leakage of small asset amounts or core functionality of the protocol is affected.
- Low - Any kind of unexpected behaviour that is non-critical.

4.2 Likelihood

- High - Almost certain to happen, easy to perform, or not easy but highly incentivized.

- Medium - Only conditionally possible or incentivized, but still relatively likely.
- Low - Involves too many or unlikely assumptions with little-to-no incentive.

4.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

5 Executive Summary

Over the course of the security review, Phi Labs engaged with MrPotatoMagic to review the phi-token-contract protocol and the phi-staking protocol. In this period of time, a total of 76 issues were found. During the review, two public audit reports were disclosed by the Phi Labs team to make the review process efficient and avoid duplication of issues. The findings from the two audit reports have been excluded from this report and are treated as known.

Summary

Project Name	Phi Token Contract
Repository	phi-token-contract, phi-staking
Commit & Pull request	c530931....6369, update-sweep
Protocol Type	Yield, SocialFi
Review Timeline	July 1st 2025 - July 31st 2025
One month fix period	July 31st 2025

Scope

- PhiEthImpl.sol
- AerodromeSwapper.sol
- BoardHook.sol
- FeeDistributor.sol
- PoolSwap.sol
- PhiEthDeployer.sol
- BasePhiEthStrategy.sol
- PhiStaking - PR update-sweep
- MorphoVaultStrategyImpl.sol
- PhiEthHooks.sol
- BoardManager.sol
- ReferralEscrow.sol
- Board.sol
- FeeValidationLib.sol
- FairLaunch.sol
- PhiStaking - Phi.sol
- AaveV3StrategyImpl.sol
- PremineZap.sol
- DynamicFeeCalculator.sol
- TokenFeePool.sol
- PhiEthFactory.sol
- StrategyDeployer.sol
- MuseManager.sol

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	5	5	0
High Risk	5	5	0
Medium Risk	21	11	10
Low Risk	23	17	6
Non-Critical	22	14	8
Total	76	52	24

6 Findings

6.1 Critical Risk

6.1.1 PhiEth creator can steal user deposited funds through Morpho Strategy

Context: MorphoVaultStrategyImpl.sol#L114
Impact: High
Likelihood: High

Description: When ETH is deposited into the Morpho strategy, depending on the rebalance threshold, certain amount of ETH remains in the PhiEthImpl contract while the remaining is converted to LST. For the Morpho strategy, this means depositing into the morpho vault and the strategy receiving morpho vault shares in return.

The issue currently is that the Morpho strategy allows the PhiEth owner/creator to call claimRewards() with arbitrary parameters. Here's what a malicious phiEth creator can do to steal the morpho shares sitting in the contract:
- Call claimRewards() and pass in reward as the morpho share token
- Pass in some random claimable and proof value. For distributor, the phiEth creator passes their own malicious contract that simply does nothing in its claim() function and returns the claimedAmount. This claimedAmount value returned is the amount of morpho shares the creator is draining.
- Following this, the safeTransfer occurs, which transfers the morpho shares to the yield receiver..

```
function claimRewards(IMorphoVaultStrategy.RewardClaimData[] calldata
rewardClaimData)
    external
    onlyPhiEthOwner
    whenInitialized
    returns (address[] memory rewardTokens, uint256[] memory amounts)
{
.... ....
```

```
uint256 claimedAmount = urd.claim(
    address(this), // account (this contract)
    claimData.reward, // reward token address
    claimData.claimable, // total claimable amount from API
    claimData.proof // merkle proof from API
);
// Transfer claimed rewards to yield receiver
if (claimedAmount > 0) {
    SafeERC20.safeTransfer(IERC20(claimData.reward), yieldReceiver,
    claimedAmount);
}
.... ....
```

Recommendation: Ensure reward token cannot be the Morpho share token. Alternatively, if extra precaution is required, consider validating the parameters with a signature provided by an authorized phiSigner entity.

6.1.2 Creator performing premine-zap unintentionally pre-closes fair launch positions

Context: BoardHook.sol#L370
Impact: High
Likelihood: High

Description: Currently in the BoardHook contract, it is possible to schedule fair launch to a specific time in the future. To ensure this is adhered to, the validation below has been introduced in the beforeSwap() hook's else block. However, an edge case missed out on is the premine zap swap, which atomically swaps tokens at the time of the board creation. This mechanism is implemented to ensure board creators can purchase their board tokens. Since for this case, we enter the if block, publphiesAt being greater than block.timestamp is overlooked.

```
if (_premineInfo.blockNumber == block.number && _p.amountSpecified
== _premineInfo.amountSpecified) {
        emit PoolPremine(poolId, _premineInfo.amountSpecified);
        _premineInfo.blockNumber = 0;
} else {
        // If the timestamp has not yet passed, then we revert
        if (_publiphiesAt > block.timestamp) revert
        TokenNotPubliphied(_publiphiesAt);

        // Remove the schedule timestamp to prevent future checks
        delete publiphiesAt[poolId];
        }
```

Followingly the code snippet below is run in the function _handleFairLaunchSwap(). Since we're not in the fair launch window, the position closes. This means the whole fair launch would never occur and fails. The root cause here is that the check considers us being out of fair launch window not only after the end of the fair launch but also before. Positions should only be closed after a fair launch window ends not before it even starts.

```
        if (!fairLaunchInfo.closed) {
            bool phiEthIsZero = currentPhiEthToken == Currency.unwrap
            (_key.currency0);


            /**
             * [FairLaunch] If the FairLaunch window has ended, but our position
             is still open, then we
             * need to close the position.
             */
            if (!fairLaunch.inFairLaunchWindow(poolId)) {
                fairLaunch.closePosition({
                    _poolKey: _key,
                    _tokenFees: _poolFees[poolId].amount1,
                    _nativeIsZero: phiEthIsZero
                });
            } else {
                beforeSwapDelta_ = _processFairLaunchSwap(poolId, _key, _p,
                _sender, _hookData, phiEthIsZero);
            }
        }
```

Recommendation: Consider closing positions only after a fair launch window ends not before it even starts.

6.1.3 Attacker can force all fair launches to fail by purchasing all tokens and selling them

Context: BoardHook.sol
Impact: High
Likelihood: High

Description: In the current fair launch mechanism through the board hook, whether a whitelist is used or not, an attacker can perform the following actions to close all fair launches:

- Buy all the fair launch tokens using the respective PhiEth token

- This will close the position and LP it into the PoolManager as observed in the FairLaunch contract
- Sell all the fair launch tokens by swapping from the board token to the PhiEth token. The hooks allow this now since the fair launch position is considered close.

The only cost to the attacker here would be the gas and fees incurred.

Recommendation: It is recommended to restrict the amount each user can purchase by implementing a default fair launch whitelist.

6.1.4 Attacker can steal all fair launch supply at no cost

Context: FairLaunch.sol#L285
Impact: High
Likelihood: High

Description: An attacker can steal all fair launch supply at no cost due to rounding down in fillFromPosition() in FairLaunch.sol. This can be done by passing in tokensOut (_p.amountSpecified) to be an extremely high value (needs to be atleast info.supply * 1e18 + 1) such that in the calculations below, we round down to 0 due to denominator being greater than numerator. This means ethIn will be set to 0 and tokensOut will be set to maxSupply.

```
if (tokensOut > info.supply) {
        /* Calculate the percentage of tokensOut relative to the threshold
        and reduce the ethIn value by the same amount. There may be some
        slight
        accuracy loss, but it's all good. */
        uint256 percentage = info.supply * 1e18 / tokensOut;
        ethIn = (ethIn * percentage) / 1e18;

        // Update our `tokensOut` to the supply limit
        tokensOut = info.supply;
    }
```

Recommendation: Consider charging a minimum amount in such a case.

6.1.5 Board tokens can be permanently locked in BoardHook.sol contract

Context: BoardHook.sol#L189
Impact: High

Likelihood: High

Description: If both initialTokenFairLaunch and premineAmount is 0, the board tokens would be locked in BoardHook contract since BoardManager mints 69 billion tokens to the contract. This is in case the user does not want to hold a fair launch i.e. initialTokenFairLaunch is 0. The tokens are locked as swaps would revert due to _beforeSwap() not considering a scenario where fair launch does not occur. It would fail at the closePosition() step as there was no fair launch position created in the first place when publiphi() was called.

```
        if (_p.initialTokenFairLaunch != 0) {
            fairLaunch.createPosition(
                poolId,
                _p.phiEthToken,
                initialTick,
                _p.publiphiAt > block.timestamp ? _p.publiphiAt :
                block.timestamp,
                _p.initialTokenFairLaunch
        );
```

Recommendation: Consider implementing a strict check to always hold a fair launch with non-zero amount of tokens.

6.2 High Risk

6.2.1 Attacker can block creators from changing strategies through minimal donations

Context: PHIETHImpl.sol#L320
Impact: Medium
Likelihood: High

Description: Function changeStrategy() in PhiEthImpl allows PhiEth creators to change strategies. However, during this process, the issue is that someone can frontrun a creator's changeStrategy() call and deposit 1 wei or dust morpho/aave share tokens in the strategy. Due to this, the check evaluates to true and would revert.

```
    if (address(strategy) != address(0) && strategy.balanceInETH() != 0) {
            revert CurrentStrategyHasBalance();
        }
```

Recommendation: Implement a dust threshold that can ignored when changing strategies.

6.2.2 Attacker can manipulate reserves on Aerodrome pools to cause slippage on withdrawAndSwap()

Context: AerodromeSwapper.sol#L72
Impact: Medium
Likelihood: High

Description: The function withdrawAndSwap() tries to perform a slippage check but fails since the return value from getAmountsOut() can be manipulated.

```
uint256[] memory expectedAmounts = aeroRouter.getAmountsOut(_amount, routes);

uint256 minAmountOut = expectedAmounts[1] * 99 / 100;
```

This is because the _getAmountOut() call to the Pool.sol contract (called through the aeroRouter) uses the reserves directly to calculate the amount of tokens to return. Since reserves are manipulatable, minAmountOut is eventually also affected and hence does not offer slippage protection when performing the swap through the router.

```
function _getAmountOut(
    uint256 amountIn,
    address tokenIn,
    uint256 _reserve0,
    uint256 _reserve1
) internal view returns (uint256) {
    if (stable) {
        uint256 xy = _k(_reserve0, _reserve1);
        _reserve0 = (_reserve0 * 1e18) / decimals0;
        _reserve1 = (_reserve1 * 1e18) / decimals1;
        (uint256 reserveA, uint256 reserveB) = tokenIn == token0 ?
        (_reserve0, _reserve1) : (_reserve1, _reserve0);
        amountIn = tokenIn == token0 ? (amountIn * 1e18) / decimals0 :
        (amountIn * 1e18) / decimals1;
        uint256 y = reserveB - _get_y(amountIn + reserveA, xy, reserveB);
        return (y * (tokenIn == token0 ? decimals1 : decimals0)) / 1e18;
    } else {
        (uint256 reserveA, uint256 reserveB) = tokenIn == token0 ?
        (_reserve0, _reserve1) : (_reserve1, _reserve0);
        return (amountIn * reserveB) / (reserveA + amountIn);
    }
}
```

Recommendation: Consider using a time-weighted average price (TWAP) for slippage with configurable parameters.

6.2.3 Unspent swap tokens are locked in ReferralEscrow.sol

Context: ReferralEscrow.sol#L158
Impact: Medium
Likelihood: High

Description: Function claimBoardAndSwap() allows users to claim and swap tokens. The tokens are swapped from board tokens to PhiEth using the PoolSwap.sol contract. The issue currently is that it is possible for less board (input) tokens to be consumed by the swap. This can also be observed through the two checks below which allow this to occur in PoolSwap.sol. Due to this, these tokens are permanently locked in the ReferralEscrow.sol contract.

```
        require(
                deltaAfter0 >= data.params.amountSpecified,
                "deltaAfter0 is not greater than or equal to
                data.params.amountSpecified"
                );

        require(
                deltaAfter1 >= data.params.amountSpecified,
                "deltaAfter1 is not greater than or equal to
                data.params.amountSpecified"
                );
```

Recommendation: Any unspent tokens should be kept allocated to the user for direct claims.

6.2.4 Morpho withdrawal can fail due to rounding issue in max withdrawal check

Context: MorphoVaultStrategyImpl.sol#L169
Impact: Medium
Likelihood: High

Description: The following code is from the function _withdrawFromMorpho(). It ensures that the user can withdraw more assets than what their shares are worth.

```
uint256 maxWithdrawable = morphoVault.maxWithdraw(address(this));
    if (amount > maxWithdrawable) {
        revert ExceedsMaxWithdrawable(amount, maxWithdrawable);
    }
```

However, the issue is that maxWithdraw() may return lower than the actual amount of assets that can be withdrawn (source: MetaMorpho contract). Due to this, it is possible for withdrawals to fail in the following scenarios::

- Last user is burning all PhiEth and withdrawing from the strategy
- All users have withdrawn and creator attempts to harvest remaining profits from the strategy
- When all tokens are attempted to be withdrawn during unwinding.

```
/// @inheritdoc IERC4626
/// @dev Warning: May be lower than the actual amount of assets that can
be withdrawn by `owner` due to conversion
/// roundings between shares and assets.
function maxWithdraw(address owner) public view override returns
(uint256 assets) {
    (assets,,) = _maxWithdraw(owner);
}
```

Recommendation: Consider removing the check as it is unnecessary. If the user does not have enough shares to burn, the metamorpho contract would simply revert.

6.2.5 Extremely low INCREASE_TOKEN_VOLUME_THRESHOLD and MAX_VOLUME_THRESHOLD in DynamicFeeCalculator.sol leads to maximum fees

Context: DynamicFeeCalculator.sol#L300
Impact: Medium
Likelihood: High

Description: In the dynamic fee calculator, INCREASE_TOKEN_VOLUME_THRESHOLD = 0.1 ether and MAX_VOLUME_THRESHOLD = 20 ether. The issue with these values is that the volume is always considered in the board token as seen in the snippet below. This means that if the board token is of low-value, 0.1e18 and 20e18 i.e. 0.1 board tokens and 20 board tokens would be an extremely small value. In which case, the fee would always be maximum.

```
        int256 deltaVolume =
            int256(Currency.unwrap(_key.currency0) == phiEthToken ?
            _delta.amount1() : _delta.amount0());
```

Recommendation: Consider adjusting INCREASE_TOKEN_VOLUME_THRESHOLD and MAX_VOLUME_THRESHOLD to more suitable values that will not result in instant maximum fees.

6.3 Medium Risk

6.3.1 Changing strategies will never be possible due to initialization check

Context: PHIETHImpl.sol#L317
Impact: Medium
Likelihood: Medium

Description: When strategies are being changed, it checks it the strategy is initialized. However, currently when strategies are deployed through the PhiEthFactory =¿ PhiEthDeployer =¿ StrategyDeployer execution path, they are initialized by default as seen here. Due to this, it is never possible to use any other strategy than the one used during deployment.

```
function changeStrategy(IBasePhiEthStrategy strategy_) external override
onlyCreator {
      if (strategy_.isInitialized()) {
          revert StrategyAlreadyInitialized();
      }
```

Additionally, the phiDeployments mapping from PhiEthFactory.sol would still be storing out of date strategy even if strategy is changed in PhiEthImpl.

Recommendation: Consider removing the initialized check and update the phiDeployments mapping in the PhiEthFactory contract.

6.3.2 Changing strategies is not possible due to external initialize call

Context: PHIETHImpl.sol#L329
Impact: Low
Likelihood: High

Description: When additional strategies are deployed through the PhiEthDeployer's deployStrategy() function, it initializes the strategy on deployment itself.

Due to this, when this initialize call is attempted in changeStrategy() in PhiEthImpl, the call reverts since the strategy is already initialized.

```
    IBasePhiEthStrategy(strategy_).initialize(address(this));
```

Recommendation: Consider removing the initialize call. Additionally it is not recommended to perform initializations in such a non-atomic manner for strategies since anyone can frontrun and initialize the strategy before.

6.3.3 Attacker can DOS publiphi by frontrunning boardIds

Context: BoardHook.sol#L123
Impact: Medium
Likelihood: Medium

Description: When a board is being created, it checks whether the attestBoardId is used or not. Since this is passed as a parameter, an attacker can frontrun and use it up. This although would be disincentived if the publiphi fee is non-zero.

```
    function publiphi(IBoardHook.PubliphiParams calldata _params)
        external
        override
        onlyBoardHook
        returns (address board_, uint256 counter_)
    {
        if (boardCreated[_params.attestBoardId]) revert
        BoardAlreadyExists(_params.attestBoardId);
```

Recommendation: A counter based approach could be used to ensure board uniqueness. Alternatively, always keep the publiphi fee non-zero to disincentivize this attack even though this would still mean the attack is possible.

6.3.4 Fees are captured twice in both afterSwap and beforeSwap hooks

Context: BoardHook.sol#L422
Impact: Medium
Likelihood: Medium

Description: Fees are captured and distributed twice in the BoardHook.sol contract. Once in the beforeSwap() hook and another in the afterSwap() hook.

```
uint256 swapFee = _captureAndDepositFees(_key, _p, _sender, swapAmount,
_hookData);
```

Recommendation: Consider removing one of the fee captures to prevent fees from being ditributed twice.

6.3.5 Attacker can steal creator's premine zap if creator directly uses publiphi on BoardHook contract

Context: BoardHook.sol#L123
Impact: High
Likelihood: Low

Description: Currently creators are allowed to publiphi through the premine-zap contract or directly through the BoardHook contract as observed by the check below. However, performing a direct publiphi through the board hook contract would pose a frontrunning risk to creators who attempt to perform a premine swap in a separate transaction in the same block. Since anyone can frontrun the user's swap transaction, this would mean that the premine amount can be stolen.

```
if (msg.sender != address(premineZap) && msg.sender != _p.creator) {
        revert UnauthorizedPubliphi(msg.sender);
    }
```

Recommendation: If msg.sender is the _p.creator, ensure premine amount is zero to disallow premine swaps in this manner. If the option is to be kept open, this finding should be be acknowledged.

6.3.6 Missing slippage protection in PoolSwap.sol

Context: PoolSwap.sol
Impact: Low
Likelihood: High

Description: Contract PoolSwap.sol does not offer protection against slippage to its callers/users. Let's look at it in every instance:

- This if block is entered when the swap is from token0 to token1. If amountSpecified is less than 0, it is considered as exactInput from token0 to token1. This means that while the amount of tokens being used on input is fixed, there is no slippage protection on the amount of output tokens received. We can see such protection being added in

the V4 Router contract here. Currently, it is only ensured that deltaAfer1 is greater than equal to 0.

```
// Sense checking of the request for safety
if (data.params.zeroForOne) {
    if (data.params.amountSpecified < 0) {
        // exact input, 0 for 1
        require(
            deltaAfter0 >= data.params.amountSpecified,
            "deltaAfter0 is not greater than or equal to
            data.params.amountSpecified"
        );

        require(delta.amount0() == deltaAfter0, "delta.amount0() is not
        equal to deltaAfter0");
        require(deltaAfter1 >= 0, "deltaAfter1 is not greater than
        or equal to 0");
```

- In this instance, it is an exactOutput swap from token0 to token1. The issue here is that the input token is only checked to be ¡= 0. This means the actual amount of tokens required can be much more than intended. This won't be an issue since if the user has approved only a specific amount of tokens to PoolSwap.sol to settle, the call would revert if more than approved tokens are attempted to be settled. However, if infinite approval is provided, it can be problematic. This check has been implemented in the V4 Router as well here.

```
if (data.params.zeroForOne) {
    if (data.params.amountSpecified < 0) {
        ...
    } else {
        // exact output, 0 for 1
        require(deltaAfter0 <= 0, "deltaAfter0 is not less than
        or equal to zero");
        require(delta.amount1() == deltaAfter1, "delta.amount1() is
        not equal to deltaAfter1");

        require(
            deltaAfter1 <= data.params.amountSpecified,
            "deltaAfter1 is not less than or equal to
            data.params.amountSpecified"
        );
    }
```

- This is the same code snippet from instance 2. However, we also have missing slippage protection for the output delta since as discussed with the uniswap team, amountSpecified is the max amount of output tokens desired, which can be less due to liquidity/market constraints.

```
    if (data.params.zeroForOne) {
        if (data.params.amountSpecified < 0) {
            ...
        } else {
            // exact output, 0 for 1
            require(deltaAfter0 <= 0, "deltaAfter0 is not less than or
            equal to zero");
            require(delta.amount1() == deltaAfter1, "delta.amount1() is
            not equal to deltaAfter1");

            require(
                deltaAfter1 <= data.params.amountSpecified,
                "deltaAfter1 is not less than or equal to
                data.params.amountSpecified"
            );
        }
```

Same applies for exactInput and exactOutput swaps from token 1 to token0 in the else block here.

Recommendation: Consider implementing configurable slippage settings for different PoolKeys.

6.3.7 Missing validation for PhiEth allows anyone to use a malicious/non-whitelisted PhiETH token when deploying strategies

Context: PhiEthDeployer.sol#L97
Impact: Medium
Likelihood: Medium

Description: When a strategy is being deployed using deployStrategy(), it is not ensured the the PhiEth token being passed in the parameter is a valid phi eth token in the factory and is whitelisted. Due to this, a malicious phi eth token that has been removed from the whitelist can still continue deploying strategies.

```
function deployStrategy(
    address phiEthToken,
    uint8 strategyType
)
```

```
        external
        returns (address strategy)
    {
```

Recommendation: Consider adding a check to ensure the PhiETH token is whitelisted.

6.3.8 Whitelisting in paused state is not enforced correctly

Context: Phi.sol
Impact: Low
Likelihood: High

Description: The following code snippet is implemented to only allow transfers between whitelisted addresses during the paused state. However, currently it is possible to bypass the whitelist due to the incorrect use of double OR operator instead of &&. For example:

- Alice is whitelisted and Bob is not
- Alice can transfer tokens from herself to Bob (non-whitelisted recipient address)
- Alternatively, non-whitelisted Bob can transfer tokens to a whitelisted Charlie recipient address.

```
    if (paused()) {
            require(isAllowedFrom[from] || isAllowedTo[to], TransferPaused());
        }
```

Recommendation: Use && instead of double OR operator.

6.3.9 Rebalance ratio not respected when rebalance threshold is increased

Context: PHIETHImpl.sol#L297
Impact: Low
Likelihood: High

Description: PhiEthImpl allows the creator to increase or decrease the rebalance threshold. For example, if rebalance ratio is 10%, this means 10% of tokens (let's say 10 tokens) remain in the PhiEthImpl contract while remaining (90 tokens) are deposited into strategies to earn yield. However, if the threshold is updated to 20%, the ratio is not maintained since there are more tokens in the strategy and less in the contract.

```
function setRebalanceThreshold(uint256 rebalanceThreshold_) external
override onlyCreator {
    if (rebalanceThreshold_ > MAX_REBALANCE_THRESHOLD) revert
    RebalanceThresholdExceedsMax();
    rebalanceThreshold = rebalanceThreshold_;
}
```

Recommendation: If rebalance threshold is increased, consider pulling out tokens from the strategy to maintain the rebalance ratio.

6.3.10 Possibility of stuck Morpho reward tokens if reward token is fee-on-transfer

Context: MorphoVaultStrategyImpl.sol#L114
Impact: Medium
Likelihood: Medium

Description: When claimRewards() is called on the morpho strategy contract, it makes an external call to the claim() function on the URD contract. This returns the claimedAmount value, which is then used to transfer out the reward tokens to the yield receiver. However, if the reward token is a fee-on-transfer token, the tokens received by the contract would be slightly lesser than the claimedAmount return value. Due to this, claims will fail as the code will attempt to transfer out more tokens than present in the contract.

```
        uint256 claimedAmount = urd.claim(
            address(this), // account (this contract)
            claimData.reward, // reward token address
            claimData.claimable, // total claimable amount from API
            claimData.proof // merkle proof from API
        );

        // Transfer claimed rewards to yield receiver
        if (claimedAmount > 0) {
             SafeERC20.safeTransfer(IERC20(claimData.reward),
             yieldReceiver, claimedAmount);
        }
```

Recommendation: Consider checking the pre-balance before the claim() and post-balance after the claim(). Use the difference to transfer the reward tokens.

6.3.11 Missing access control on setFactory() enables frontrunning initialization

Context: PhiEthDeployer.sol#L51

Impact: Low
Likelihood: High

Description: The below function is present in PhiEthDeployer.sol. However, the function currently does not have access control which allows anyone to call the function directly and set the factory. Due to this, one can set this to a malicious implementation.

```
function setFactory(address _factory) external {
    if (factory != address(0)) revert PhiEthErrors.FactoryAlreadySet();
    if (_factory == address(0)) revert PhiEthErrors.ZeroReceiverAddress();
    factory = _factory;
}
```

Recommendation: Implement access control on the function or simply set the factory in the constructor.

6.3.12 Inconsistent PhiEthDeployer update logic across contracts

Context: StrategyDeployer.sol#L110
Impact: Low
Likelihood: High

Description: In StrategyDeployer.sol, function setPhiEthDeployer() allows the owner to update the phi eth deployer address. However, in PhiEthFactory.sol, there is no functionality present that allows updating the PhiEthDeployer. Due to this, the update logic for PhiEthDeployer wil never work.

Recommendation: Implement a function in PhiEthFactory.sol that allows the owner to update the phiEthDeployer address. Alternatively, if PhiEthFactory updates are not intended to be supported, setPhiEthDeployer() can be removed from the StrategyDeployer.sol contract.

6.3.13 Ranking of existing second and third place top contributors does not increase

Context: MuseManager.sol#L209
Impact: Low
Likelihood: High

Description: In function _updateTopContributors(), the following for loop exists. The issue is that if a contributor in the second or third place increases their inspiration count, they are not updated in the top contributors list since we return early. This is problematic since the topContributor mapping now stores the highest ranked address while topContributors mapping stores incorrect data.

```
        for (uint256 i = 0; i < 3; i++) {
            if (topContributors[i] == contributor) return;
        }
```

Recommendation: Consider adding logic to update the topContributors mapping for the second and third ranking contributors.

6.3.14 Contributor ranking logic incorrectly switches second rank instead of 3rd rank

Context: MuseManager.sol#L225
Impact: Low
Likelihood: High

Description: In the function _updateTopContributors(), the lowest index is not retrieved. For example, if rank 2 and 3 have the lowest counts and are equal, rank 2 is considered as lowest count instead of rank 3. Due to this, the contributor in the second place is replaced instead of third.

```
        for (uint256 i = 0; i < 3; i++) {
            if (topContributors[i] == address(0)) {
                topContributors[i] = contributor;
                emit TopContributorsUpdated(boardAddress, contributor,
                contributorCount, i + 1, epoch);
                return;
            }

            uint256 count = inspirationCounts[boardAddress][epoch]
            [topContributors[i]];

            if (count < lowestCount) {
                lowestCount = count;
                lowestIndex = i;
            }
        }

        // Replace lowest inspiration contributor if new contributor
        has higher inspiration count
        if (contributorCount > lowestCount && lowestIndex < 3) {
            topContributors[lowestIndex] = contributor;
            emit TopContributorsUpdated(boardAddress, contributor,
            contributorCount, lowestIndex + 1, epoch);
        }
```

Recommendation: We should use less than equal to operator instead in the check if (count less than lowestCount)

6.3.15 Fees in feeSplit() are incorrectly calculated

Context: FeeDistributor.sol#L162
Impact: Low
Likelihood: High

Description: Let's say totalAmount = 100 and fees are 20%, 10%, 10% for swapper, inspirer and protocol respectively. What happens is swapperFee_ would evaluate as 20 tokens. In swapper if block in the code snippet, since _amount is deducted to 80 tokens, the inspirer fee would evaluate as 10% of 80 instead of 10% of 100, which means it amounts to 8 tokens. In inspirer block _amount is reduced to 80-8 = 72. Now for protocol fees, it evaluate as 10% of 72 = 7.2 tokens instead of 10% of 100 = 10 tokens. And the remaining goes to the creator. This means the inspirer and protocol are underpaid. Swapper gets the right amount and creator gets overpaid.

```
        for (uint256 i = 0; i < 3; i++) function feeSplit(
        PoolId _poolId,
        uint256 _amount
    )
        public
        view
        returns (uint256 swapperFee_, uint256 inspirer_, uint256 protocol_,
        uint256 creator_)
    {
        // Check if we have a pool overwrite for the FeeDistribution
        FeeDistribution memory _poolFeeDistribution = getPoolFeeDistribution
        (_poolId);

        if (_poolFeeDistribution.swapper != 0) {
            swapperFee_ = _amount * _poolFeeDistribution.swapper /
            ONE_HUNDRED_PERCENT;
            _amount -= swapperFee_;
        }

        if (_poolFeeDistribution.inspirer != 0) {
            inspirer_ = _amount * _poolFeeDistribution.inspirer /
            ONE_HUNDRED_PERCENT;
            _amount -= inspirer_;
        }

         if (_poolFeeDistribution.protocol != 0) {
            protocol_ = _amount * _poolFeeDistribution.protocol /
            ONE_HUNDRED_PERCENT;
```

```
            _amount -= protocol_;
        }

        // The creator will receive the remaining allocation
        creator_ = _amount;
    }
```

Recommendation: Consider splitting fees as per the fee percents.

6.3.16 Only first swap in block receives decreased volume based fee

Context: DynamicFeeCalculator.sol
Impact: Low
Likelihood: High

Description: In function calculateVolumeBasedFee(), we enter the else block if the rolling fee window duration has not passed. This is determined based on the timeElapsed that uses lastFeeIncreaseTime, which is updated to block.timestamp every time the currentFeeScaled variable increases.

Currently the issue in the else block is that, only first swap in a block receives decreased volume based fee. This is because if volume is increased, lastFeeIncreaseTime is updated by this first swap itself. This means any subsequent swaps in the same block would receive currentFeeScaled in its entirety since timeElapsed would be 0. This might be intended however this behaviour is something to be aware of.

```
    function calculateVolumeBasedFee(PoolId _poolId) internal view
    returns (uint256) {
        PoolInfo storage poolInfo = poolInfos[_poolId];
        uint256 timeElapsed = block.timestamp - poolInfo.lastFeeIncreaseTime;

        // If there's no fee increase within the last rolling fee
        window, fees return back to the minimum
        if (timeElapsed >= ROLLING_FEE_WINDOW_DURATION) {
            return MINIMUM_FEE_SCALED;
        } else {
            // Calculate the fee decrease based on the time elapsed
            since the last fee increase
            uint24 currentFeeScaled = poolInfo.currentFeeScaled;

            // Fee should linearly tend towards the minimum fee over
            the rolling fee window

            uint256 feeDecreaseScaled =
                ((currentFeeScaled - MINIMUM_FEE_SCALED) * timeElapsed) / ROLLING_FEE_WINDOW_DU
```

```
        return currentFeeScaled - feeDecreaseScaled;
    }
}
```

Recommendation: Consider acknowledging this behaviour.

6.3.17 Malicious user can dilute fees by providing 20 gwei priority fee at the start of block

Context: DynamicFeeCalculator.sol
Impact: Low
Likelihood: High

Description: Function calculateGasBasedFee() calculates the fees based on the transaction priority fees in the current transaction relative to the highest transaction priority fee from that block. The issue is that an attacker can simply make a 20 gwei priority fee transaction at the start of the block. This would set topPriorityFee to 20 gwei, which means any following swaps in that block would be cheap and hence diluted.

```
uint256 gasBasedFeeScaled = (maxGasFee * txPriorityFee) / topPriorityFee;
```

Recommendation: Consider acknowledging this issue or implementing a gradual increase for the top priority fee increase.

6.3.18 Possible for gasBasedFeeScaled to be greater than maxGasFee in DynamicFeeCalculator

Context: DynamicFeeCalculator.sol
Impact: Low
Likelihood: High

Description: Function calculateGasBasedFee() calculates the gas based fee component. However, the issue is that gasBasedFeeScaled can be greater than maxGasFee. Let's say if topPriorityFee = 2 gwei and now in same block but different transaction someone provides txPriorityFee = 12 gwei, this means that gasBasedFeeScaled would evaluate as 20000 * 12 gwei / 2 gwei = 20000 * 6 gwei. This means that the gasBasedFeeScaled can be 6 times maxGasFee.

```
    uint256 gasBasedFeeScaled = (maxGasFee * txPriorityFee) / topPriorityFee;
    return gasBasedFeeScaled < minGasFee ? minGasFee : gasBasedFeeScaled;
```

Recommendation: If gasBasedFeeScaled is greater than maxGasFee, limit it to maxGasFee as done for minGasFee.

6.3.19 calculateGasBasedFee() incorrectly calculates gas based fee component for first swap in a block

Context: DynamicFeeCalculator.sol
Impact: Low
Likelihood: High

Description: In the function calculateGasBasedFee(), the topPriorityFee is retrieved from the lastBlockSeen. If topPriorityFee is from a past block after which it has not been updated at all, then the calculation here would be inaccurate since txPriorityFee would be from this block but topPriorityFee is from a past block. This means the gas based fee scaled value calculates the current transaction's priority fee to an older top priority fee from a past block, which is not accurate.

```
        uint256 gasBasedFeeScaled = (maxGasFee * txPriorityFee) / topPriorityFee;
        return gasBasedFeeScaled < minGasFee ? minGasFee : gasBasedFeeScaled;
```

Recommendation: Consider acknowledging this behaviour or implement a fix for it

6.3.20 Attacker can use flashswaps to increase fees for users

Context: DynamicFeeCalculator.sol
Impact: Low
Likelihood: High

Description: When a flash swap is performed, the trackSwap() function will be called through the board hook contract to track the newly updated fee scale based on the volume of the swap. This is stored in accumulatorWeightedVolume, which determines the new fee scaled value. The issue with the model in DynamicFeeCalculator currently is that it is extremely easy to inflate fees for users. Due to this, the dynamic fee calculator becomes redundant in the functionality to attempts to offer.

Recommendation: Consider acknowledging this behaviour or implementing a fix for it.

6.3.21 Fee decay does not factor in frequency of swaps

Context: DynamicFeeCalculator.sol
Impact: Low
Likelihood: High

Description: In function trackSwap(), for high volume/activity pools, decay will never occur. This is because if there are frequent swaps/activity in the pool, timeElapsed will

never be big enough (since accumulatorLastUpdateTime is reset to block.timestamp at each swap). This means that even if let's say the transaction volume of older swaps is higher than the current ones, the fees would still be high since accumulatorWeightedVolume would still contain old volume data. The issue here is that the logic currently does not consider in the factor of frequency i.e. if swaps are more frequent, older swap volumes still hold weight.

Recommendation: After X number of swaps are completed within a certain time window, consider increasing the decay to account for frequency.

6.4 Low Risk

6.4.1 Remove console.log statement

Description: The following statement in PhiEthImpl is only supposed to be used for testing purposes. Consider removing it before production.

```
import { console } from "forge-std/console.sol";
```

Recommendation: Remove the console.log import

6.4.2 Incorrect publiphiAt timestamp emitted on board creation

Description: In BoardManager.sol, publiphi() function is called from the BoardHook. If publiphiAt is less than block.timestamp, the event below will be emitted with the timestamp in the past while the actual publiPhiAt set in the the board hook would be block.timestamp.

```
        emit BoardCreated(
            counter_,
            board_,
            _params.attestBoardId,
            _params.creator,
            _params.inspirer,
            _params.initialTokenFairLaunch,
            _params.premineAmount,
            _params.publiphiAt
        );
```

Recommendation: If publiphiAt is less than block.timestamp, emit block.timestamp.

6.4.3 Missing address(0) check when adding and removing a router

Description: In BoardManager.sol, there are missing address(0) checks when adding and removing a router. While this does not pose a risk, it is recommended to add the checks to prevent any unwanted scenarios.

```
function addRouter(address _router) public onlyOwner {
    verifiedRouters[_router] = true;
    emit RouterAdded(_router);
}

function removeRouter(address _router) public onlyOwner {
    verifiedRouters[_router] = false;
   emit RouterRemoved(_router);
}
```

Recommendation: Add address(0) checks to the respective functions.

6.4.4 Missing event emission in setGasFeeParameters() in DynamicFeeCalculator.sol

Description: Consider emitting an event when the gas based parameters in the dynamic fee calculator are updated by the owner.

```
    // Update pool settings
    PoolInfo storage poolInfo = poolInfos[_poolId];
    poolInfo.minGasFee = _minGasFee;
    poolInfo.maxGasFee = _maxGasFee;
    poolInfo.weightVolume = _volumeWeight;
    poolInfo.weightGas = _gasWeight;
```

Recommendation: Emit an event on gas based parameter updates.

6.4.5 Whitelist merkle root cannot be updated by creator

Description: Once a whitelist is set during publiphi, it cannot be updated by a creator. Providing this functionality does not seem to pose a risk currently since whitelist is only used during fair launch swaps.

```
        if (_p.merkleRoot != bytes32(0)) {
            flWhitelist[poolId] = _p.merkleRoot;
        }
```

Recommendation: Consider acknowledging this or implementing the functionality recommended above.

6.4.6 Protocol fee recipient can revert to temporarily DOS publiphi

Description: When transferring ETH to the protocolFeeRecipient, it is possible for the fee recipient to revert on the ETH transfer to temporarily DOS publiphi. While the protocolFeeRecipient can be changed, this is a risk to be aware of.

```
    SafeTransferLib.safeTransferETH(protocolFeeRecipient, publiphiFee);
```

This applies to other instances as well, for example, when distributing fees and if swap is skipped in withdrawAndSwap():

```
    ISwapper(phiEthFactory.poolSwapZap()).withdrawAndSwap
    (currentPhiEthToken, swapperFee, protocolFeeRecipient);
```

Recommendation: Consider acknowledging this risk and monitor the protocol fee recipient.

6.4.7 Incorrect event emission of PoolFeesDistributed

Description: As per the event definition of PoolFeesDistributed, the 4th member should be swapperFee, 5th should be inspiredFee and 6th should be protocolFee.

```
emit PoolFeesDistributed(poolId, distributeAmount, creatorFee,
creatorFee, swapperFee, inspiredFee);
```

Recommendation: Correct the event emission members as recommended above.

6.4.8 Self referrals are possible

Description: The referrer is obtained in BoardHook._processReferrerAndDepositFees() through the hookData. However, since there is no validation on who this referrer is, the user themselves can pass in their address as the referrer address or even use their alias accounts. This might be fine but it is a scenario to be aware of.

```
SwapData memory data = abi.decode(_hookData, (SwapData));
    address referrer = data.referrer;

    // Check if we have a referrer set and send them the currency directly
    uint256 referrerFee = _distributeReferrerFees({
        _key: _poolKey,
        _swapFeeCurrency: swapFeeCurrency,
        _swapFee: swapFee_,
        _referrer: referrer
    });
```

Recommendation: Consider acknowledging this behaviour or implement logic to avoid self referrals.

6.4.9 Incorrect event emission of TokenSwapped in ReferralEscrow.sol

Description: The following event is emitted in the function claimBoardAndSwap(). However, if the swap is skipped in the function _performSwap() when the token is phiEth, it still emits the event.

```
emit TokensSwapped(msg.sender, info.token, info.amount, info.amountOut);
```

Recommendation: It is recommended to move this event emission in the performSwap() function.

6.4.10 Incorrect endsAt data in event FairLaunchEnded emission

Description: As we can see from the comment below, if block.timestamp is past the actual end time, we're supposed to backdate it and emit the actual endsAt time. However, since endsAt is marked as block.timestamp just above the comment, this would emit the current timestamp.

```
info.endsAt = block.timestamp;
info.closed = true;

    // Emit the event with the balance of the currency we hold before
    // we create a position with
    // it. We determine the end time by seeing if it has ended early,
    //or if we are past the point
    // it was meant to end then we backdate it.
    emit FairLaunchEnded(_poolKey.toId(), info.revenue, info.supply,
    info.endsAt);
```

Recommendation: Depending on the intention, either correct the event emission or improvise the comments.

6.4.11 Function claimBoardAndSwap() can revert during fair launch window

Description: Calls to claimBoardAndSwap() in ReferralEscrow.sol can revert currently since it performs a swap from the board token to the phi eth token, which is considered as selling during the fair launch window and disallowed. Even if this was allowed only for the referral escrow contract, it passes in empty proof and leafData. As it calls the first swap() function in the PoolSwap.sol contract, it would revert if there is a whitelist merkle root.

The severity is low since the fair launch is only 30 minutes. The UI can provide users trying to claim with warnings that such transactions will fail and it would be better to use the regular claimTokens() function during that period.

```
function swap(
    PoolKey memory _key,
    IPoolManager.SwapParams memory _params
)
    public
    payable
    setMsgSender
    returns (BalanceDelta)
{
    return swap(_key, _params, address(0), new bytes32[](0), "");
}
```

Recommendation: Consider acknowledging this behaviour.

6.4.12 Consider making slippage percentage configurable in AerodromeSwapper.sol

Description: Currently the withdrawAndSwap() function uses a hardcoded 1% slippage percent. However, it is recommended to keep this parameter configurable in case the slippage exceeds more than 1%.

```
uint256 minAmountOut = expectedAmounts[1] * 99 / 100;
```

Recommendation: Consider keeping the slippage percent configurable.

6.4.13 Precision loss due to rounding down in withdrawAndSwap()

Description: Currently, we multiply the expectedAmount by 99, followed by a division by 100, which can cause loss of precision due to rounding down in solidity i.e. the numerator not being a multiple of 100. For example:

- x = expectedAmounts[1] is a not a multiple of 100
- y = 99 is not a multiple of 100
- The product of x and y = z is not a multiple of 100

Since solidity does not support decimals, we have z / 100 to be value rounded down to the nearest multiple of 100.

```
uint256 minAmountOut = expectedAmounts[1] * 99 / 100;
```

Recommendation: It is recommended to use the BPS model i.e. 10000 in denominator. This way 1% can be represented as 100/10000, which can prevent precision loss slightly better.

6.4.14 No incentive for users to purchase PhiEth

Description: In PhiETHImpl, anyone can deposit their ETH to receive PhiEth and vice versa. The issue is that, any yield accrued by this, is only earned by the protocol and the yield receiver set by the phiEth creator. Due to this, there is no incentive for users to deposit and purchase PhiEth.

Recommendation: Consider sharing yield with users or acknowledging this as the design choice.

6.4.15 Missing functionality to allow users to invalidate permits

Description: The Phi.sol token contract does not allow users to increment their nonces, which allows them to invalidate existing permits.

Recommendation: Consider providing functionality that allows an user to invalidate their own permit by incrementing their nonce.

6.4.16 Consider removing redundant zero address check in mint()

Description: The zero address check on _to is not required since it is already present in the internal _mint() function.

```
function mint(address _to, uint256 _amount) public virtual override
onlyPublify {
    if (_to == address(0)) revert MintAddressIsZero();
    _mint(_to, _amount);
}
```

Recommendation: Consider removing the redundant zero address check.

6.4.17 Remove redundant functionality from Board.sol

Description: The _givePermit2InfiniteAllowance() always returns true. Since this return value is used in the allowance() and approve() functions, this means we will always enter the if conditional blocks.

```
function _givePermit2InfiniteAllowance() internal view virtual
returns (bool) {
    return true;
}

function allowance(
    address owner,
    address spender
)
    public
    view
    override(ERC20Upgradeable, IERC20)
    returns (uint256)
{
```

```
        if (_givePermit2InfiniteAllowance()) {
            if (spender == _PERMIT2) return type(uint256).max;
        }
        return super.allowance(owner, spender);
    }

    /**
     * Override to support Permit2 infinite allowance.
     */
    function approve(address spender, uint256 amount) public
    override(ERC20Upgradeable, IERC20) returns (bool) {
        if (_givePermit2InfiniteAllowance()) {

            if (spender == _PERMIT2 && amount != type(uint256).max) {
                revert Permit2AllowanceIsFixedAtInfinity();
            }
        }
        return super.approve(spender, amount);
    }
```

Recommendation: Consider removing the function and the if conditional while keeping the logic inside the if blocks in place. Example implementation:

```
    function allowance(
        address owner,
        address spender
    )
        public
        view
        override(ERC20Upgradeable, IERC20)
        returns (uint256)
    {

            if (spender == _PERMIT2) return type(uint256).max;

        return super.allowance(owner, spender);
    }

    /**
     * Override to support Permit2 infinite allowance.
     */
    function approve(address spender, uint256 amount) public
    override(ERC20Upgradeable, IERC20) returns (bool) {
```

```
        if (spender == _PERMIT2 && amount != type(uint256).max) {
            revert Permit2AllowanceIsFixedAtInfinity();
        }

    return super.approve(spender, amount);
}
```

6.4.18 Inspirations to address(0) occur on burns

Description: Function tryInspire() allows contributors to inspire recipients. However, currently inspirations to address(0) are possible on burns since the to address is the zero address.

```
function _update(address from, address to, uint256 amount) internal override
{
    super._update(from, to, amount);

    if (museManager.shouldInspire(from, amount)) {
        museManager.tryInspire(from, to, amount);
    }
}
```

Recommendation: Disallow inspirations to address(0) on burns.

6.4.19 Inspirer update logic allows creator to deprive original inspirer of fees

Description: The creator can update the inspirer to any address (an alias account or even address(0)) to deprive the original inspirer of their fees.

```
function setInspirer(address boardAddress, address newInspirer) external {
    if (boardToCreator[boardAddress] != msg.sender) revert
    OnlyBoardCreatorCanSetInspirer();
    if (newInspirer == msg.sender) revert CreatorCannotBeOwnInspirer();

    address oldInspirer = boardToInspirer[boardAddress];
    boardToInspirer[boardAddress] = newInspirer;

    emit InspirerUpdated(boardAddress, oldInspirer, newInspirer,
```

```
        block.timestamp);
    }
```

Recommendation: Consider acknowledging this risk or disallowing inspirer updates.

6.4.20 Incorrect event emission details

Description: The oldBoardManager variable is incorrect as it should store the boardManager and not the parameter _boardManager.

```
    function setBoardManager(address _boardManager) external onlyOwner {
        if (_boardManager == address(0)) revert InvalidBoardManager();

        address oldBoardManager = _boardManager;
        boardManager = _boardManager;
        emit BoardManagerUpdated(oldBoardManager, _boardManager);
    }
```

Recommendation: Use boardManager instead of _boardManager for variable oldBoardManager.

6.4.21 Incorrect fairlaunch comment in publiphi

Description: The following comment in the BoardHook.publiphi() function is incorrect since at token creation, a fair launch position is created. However, the token supply is only put into a one-sided position after closePosition() is called.

```
        /**
         * [FairLaunch] At token creation, x% of token supply is put
         into a one-sided position.
         */
```

Recommendation: Improvise the comment to reflect the correct behaviour.

6.4.22 Redundant PhiEth token whitelist functionality in PhiEthFactory.sol

Description: Currently nowhere in the codebase do we check whether a PhiEth token is whitelisted or not. For example, the claimTokens() function in ReferralEscrow.sol. While

this does not seem be to an issue, it is recommended to appropriately apply the verification checks across the codebase. Instances where the verification is not applied:

- deployStrategy() in PhiEthDeployer
- ReferralEscrow.sol PhiEth claims being allowed even when token is removed from whitelist
- PhiEthImpl deposits still being allowed even when token is removed from whitelist.

Recommendation: Consider applying verification on the respective instances and across the codebase.

6.4.23 Variable accumulatorLastUpdateTime updated even when accumulator weighted volume remains unchanged

Description: The following snippet poolInfo.accumulatorLastUpdateTime = block.timestamp; is always run when a swap occurs. However, let's assume the pool's accumulatorWeighted-Volume is type(uint256).max for a few blocks now. This means the decay would occur at a very slow rate as timeElapsed would be 0 (assuming frequent swaps occur) and the weighted volume would always be reset to poolInfo.accumulatorWeightedVolume = type(uint256).max / 1 ether;.

Recommendation: Consider acknowledging this behaviour or implementing a fix for it.

6.5 Non-Critical

6.5.1 phiEthFactory is already assigned in FeeDistributor.sol

Description: In the constructor of the BoardHook.sol contract, we assign phiEthFactory again even though it was already initialized in the FeeDistributor contract inherited by the BoardHook.sol contract.

```
phiEthFactory = PhiEthFactory(_p.phiEthFactory);
```

Recommendation: Remove one of the initializations for phiEthFactory.

6.5.2 Redundant else-if block in function modifyRevenue()

Description: Function modifyRevenue() is called from the BoardHook.

```
function modifyRevenue(PoolId _poolId, int256 _revenue) public onlyBoardHook
{
    if (_revenue < 0) {
        _fairLaunchInfo[_poolId].revenue -= uint256(-_revenue);
    } else if (_revenue > 0) {
        _fairLaunchInfo[_poolId].revenue += uint256(_revenue);
    }
}
```

However since swapFee is always negatively passed from the hook, the else if block is never reachable.

```
if (_p.amountSpecified >= 0 && swapFee != 0) {
        fairLaunch.modifyRevenue(poolId, -swapFee.toInt128());
    }
```

Recommendation: Consider removing the else-if block.

6.5.3 Missing functionality for owner to update boardManager, feeExemptions, fairLaunch and notifier contract addresses

Description: BoardHook provides functionality to update these addresses however no functionality is offerred to update boardManager, feeExemptions, fairLaunch and notifier contract addresses.

```
function setPremineZap(address _premineZap) external onlyOwner {
    premineZap = _premineZap;
}

function setInitialPrice(address _initialPrice) public onlyOwner {
    initialPrice = IInitialPrice(_initialPrice);
}

function setMuseManager(address _museManager) external onlyOwner {
    museManager = IMuseManager(_museManager);
}
```

Recommendation: Implement the respective setter functions or acknowledge this behaviour.

6.5.4 Return value of createPosition() is never used

Description: Function createPosition() in FairLaunch.sol returns the FairLaunchInfo struct created. However, this is never used in the BoardHook contract.

```
return _fairLaunchInfo[_poolId];
```

Recommendation: Consider removing the returned variable.

6.5.5 Add premine zap contract to excludedAddresses mapping in MuseManager.sol

Description: Since the premine zap contract transfers board tokens to the msg.sender, it also performs a tryInspire(). It is recommended to add it in the excludedAddresses list to skip these inspirations.

```
IERC20(_boardToken).transfer(msg.sender, _premineAmount);
```

Recommendation: Add premine zap in the excludedAddresses list to skip these inspirations.

6.5.6 Consider making swap() functions in the PoolSwap.sol contract non-payable

Description: Both the swap() functions in PoolSwap.sol can be marked as non-payable since neither does the PremineZap.sol nor the ReferralEscrow.sol use it to swap ETH.

Recommendation: Remove the payable keyword from them.

6.5.7 Deadline is not required in withdrawAndSwap()

Description: The use of + 300 is not required when providing the deadline to the aeroRouter. This is because the transaction is atomically executed contract to contract. It is recommended to use block.timestamp simply.

```
uint256[] memory amounts =
    aeroRouter.swapExactTokensForTokens(_amount, minAmountOut, routes,
    _recipient, block.timestamp + 300);
```

Recommendation: It is recommended to use block.timestamp simply.

6.5.8 Correct comment in PhiEthImpl

Description: In the comment above the initialize() function, it is mentioned that the initial strategy can be zero. However this is not true since when a PhiEth contract is deployed through the deployer, it is always accompanied with a non-zero strategy address that is initialized.

```
* @param strategy_ The initial strategy address (can be zero)
```

Recommendation: Consider removing the comment or correcting it.

6.5.9 Consider removing redundant variable from PhiEthFactory

Description: The below variable is not used in the PhiEthFactory contract. Consider removing it.

```
address public constant CREATE2_DEPLOYER =
0x4e59b44847b379578588920cA78FbF26c0B4956C;
```

Recommendation: Remove the variable CREATE2_DEPLOYER

6.5.10 Consider introducing functionality to remove strategies from whitelist

Description: Introduce functionality to remove strategy from whitelist. While this does not seem to be required currently as it does not pose a risk, it would be better to have the option.

```
function addStrategyToWhitelist(address _strategy) external {
    // Allow deployer to add automatically, owner to add manually
    require(msg.sender == address(deployer) || msg.sender == owner(),
    "Unauthorized");
    whitelistedStrategies[_strategy] = true;
    emit StrategyWhitelisted(_strategy);
}
```

Recommendation: Consider introducing functionality to remove strategies from whitelist.

6.5.11 Function setBoardImplementation() can be optimized

Description: Emitting before updating boardImplementation can help save gas since you do not need to create the memory variable oldImplementation.

Recommendation: Optimize the function as shown below:

```
function setBoardImplementation(address _newBoardImplementation) external
onlyOwner {
    if (_newBoardImplementation == address(0)) revert InvalidAddress();

    emit BoardImplementationUpdated(boardImplementation,
    _newBoardImplementation);
    boardImplementation = _newBoardImplementation;

}
```

6.5.12 Pack struct members to save gas

Description: Here is the current number of storage slots the FairLaunchInfo occupies i.e. 6 slots. Each slot in solidity can store upto 32 bytes. This way, if we move initialTick to the end, we can pack it in 1 slot. This means in total, we can save upto 1 slot. Since this struct is used in the _fairLaunchInfo mapping, it will be a useful gas optimization.

```
struct FairLaunchInfo {
    uint256 startsAt; // 1st slot (32 bytes)
    uint256 endsAt; // 2nd slot (32 bytes)
    int24 initialTick; // 3rd slot (3 bytes)
    uint256 revenue; // 4th slot (32 bytes)
    uint256 supply; // 5th slot (32 bytes)
    bool closed; // 6th slot (1 byte)
    address phiEthToken; // 6th slot (20 bytes)
}
```

Recommendation: Consider packing the struct as recommended above.

6.5.13 Consider removing redundant modifier from swap()

Description: PoolSwap.sol exposes two swap functions. The swap() function below calls the second swap() function. Since the second swap() function already has the setMsgSender

modifier applied on it, it is recommended to remove the modifier from the function below. This will save gas.

```
function swap(
    PoolKey memory _key,
    IPoolManager.SwapParams memory _params
)
    public
    payable
    setMsgSender
    returns (BalanceDelta)
{
    return swap(_key, _params, address(0), new bytes32[](0), "");
}
```

Recommendation: Consider following the recommendation provided above.

6.5.14 Remove unused variable PhiEthDeployer

Description: The following variable is not used in StrategyDeployer.sol.

```
address public immutable PhiEthDeployer;
```

Recommendation: Consider following the recommendation provided above.

6.5.15 Strategy implementations cannot be updated in strategy deployer

Description: The AAVE and Morpho implementations once set in the constructor of the StrategyDeployer.sol contract cannot be updated. While this is not an issue, it could be useful in case a new version of the implementation needs to be used. However note that this could introduce a centralization risk as a trade-off, where a malicious implementation could be set in case the permissioned entity is compromised.

```
    aaveV3Implementation = _aaveV3Implementation;
    morphoImplementation = _morphoImplementation;
```

Recommendation: Consider following the recommendation provided above or acknowledge the issue.

6.5.16 Creator cannot deploy PhiEth token again with the same name and symbol

Description: In function _deployTokenAndStrategy() in PhiEthDeployer.sol, the salt is calculated using the chainId, creator, name and symbol, which is then used to deterministically deploy a contract using the create2 opcode. However, since the salt does not include a nonce, this means the same creator cannot deploy a PhiEth token with the same name and symbol again. While this is not an issue, this behaviour is something to be aware of.

```
        bytes32 salt = keccak256(abi.encodePacked(block.chainid, params.creator,
        params.name, params.symbol));

        // Deploy phiEth token clone
        phiEthToken = payable(PHI_TOKEN_IMPL.cloneDeterministic(salt));
```

Recommendation: Consider acknowledging the issue or include a nonce in the salt calculation to resolve it.

6.5.17 Consider using named mappings in MuseManager.sol

Description: Currently the mappings in the muse manager contract are not named. It is recommended to name them to improve code readability. Below listed are only few instances, however it is recommended to follow this style guide for all contracts.

```
    mapping(address => bool) private excludedAddresses;

    // === INSPIRATION SETTINGS ===
    // Maps board address to its designated inspirer (manually set by creator)
    mapping(address => address) public boardToInspirer;

    // Maps board address to the creator who issued it
    mapping(address => address) public boardToCreator;
```

Recommendation: Consider following the recommendation provided above.

6.5.18 Typo error with inspirer instead of inspired

Description: The comment above the mapping declaration uses inspirer instead of inspired.

```
// Tracks number of unique active recipients each sender has inspirer:
board => epoch => contributor => unique_count
mapping(address => mapping(uint256 => mapping(address => uint256)))
public inspirationCounts;
```

Recommendation: Consider correcting the typo error

6.5.19 Save gas by emitting event first

Description: Function setInspirationThreshold() can be optimized to save gas. Consider using the implementation below instead of the current implementation.

```
function setInspirationThreshold(uint256 _newThreshold) external onlyOwner {
    if (_newThreshold == 0) revert InvalidThreshold();
    emit InspirationThresholdUpdated(inspirationThreshold, _newThreshold,
    block.timestamp);
    inspirationThreshold = _newThreshold;
}
```

A similar instance exists in MuseManager and setBoardManager() in MuseManager.sol.

```
    address oldTopContributor = topContributor[boardAddress][epoch];
    topContributor[boardAddress][epoch] = contributor;
    topContributorCount[boardAddress][epoch] = newCount;

    emit TopContributorUpdated(
         boardAddress, oldTopContributor, contributor, newCount,
         epoch, block.timestamp
    );
```

Recommendation: Consider following the recommendation provided above.

6.5.20 Remove redundant excludedAddresses check from tryInspire()

Description: The below check in tryInspire() is already present in shouldInspire(), which is called at the start of the function tryInspire(). Hence this piece of code can be removed.

```
if (excludedAddresses[contributor]) return false;
```

Recommendation: Consider removing the check.

6.5.21 Alias account transfers can be used to inspire

Description: In tryInspire(), a malicious user can transfer minimum inspiration threshold amount of tokens to itself through alias accounts to become the top contributor. For example,

- Alice can transfer board tokens from account 1 to 2.
- Account 2 back to account 1
- Account 1 to account 3
- Account 3 to account 1 and so on so forth.

This way, account 1 can eventually become the top contributor.

Recommendation: Consider acknowledging this risk or implement rate limit on a particular account's inspirations.

6.5.22 Incorrect comments in trackSwap()

Description: The comment below is incorrect since we're looking at volume of token boardToken from phiEthToken-boardToken pair, where boardToken can either be token 1 or token 0.

```
// Calculate the fee based on the accumulated volume of token1
```

Another instance mentions that volume should be capped at max if it exceeds total token supply. This is incorrect since the check ensures that if the volume exceeds the max volume threshold (and not total token supply), the fee will be capepd at max.

```
if (poolInfo.accumulatorWeightedVolume >= MAX_VOLUME_THRESHOLD) {
// If the volume exceeds the total token supply, the fee should be capped at
// the maximum.
```

Recommendation: Consider correcting the comments accordingly.